Module 01

Complexity & analysis

Before choosing a data structure, we need a way to compare the work it performs. Complexity analysis describes how time and memory requirements grow with input size. It lets us reason about an algorithm beyond a single benchmark on a single computer.

Thinking in terms of growth

An algorithm's running time depends on its input size, its input arrangement, and the machine executing it. Complexity analysis isolates the first two factors by counting meaningful operations.

For a list of n values, adding every value needs n additions. Doubling the list approximately doubles that work. We describe this as linear growth. A fixed amount of setup does not change the long-term growth rate.

Ask which operation dominates, how often it occurs, and what the worst plausible input looks like. This gives a more transferable comparison than a stopwatch measurement alone.

Big O is an upper bound

We write T(n) = O(f(n)) when a constant multiple of f(n) eventually bounds T(n) above. Constants and lower-order terms are ignored for this comparison. For example, 3n + 10 is O(n).

Common growth rates, from slower to faster, include O(1), O(log n), O(n), O(n log n), and O(n^2). Big O is an upper bound; it is not automatically an exact description of every input or of average performance.

For a tight asymptotic bound, Theta notation expresses both an upper and a lower bound of the same order.

Worked example: nested loops

Generated practice example: consider an outer loop that executes n times and an inner loop that executes n times for every outer iteration.

1. One outer iteration performs n inner operations.
2. There are n outer iterations.
3. The total is n x n = n^2 operations.
4. The running time is O(n^2).

Two consecutive loops of n iterations each perform 2n operations and are O(n), not O(n^2). Multiplication applies to nesting, while addition applies to consecutive work.

Memory matters too

Auxiliary space counts additional working memory beyond the input. Summing a list with one accumulator uses O(1) auxiliary space. Copying all n elements to a new list uses O(n) auxiliary space.

Recursive algorithms also consume stack frames. A recursion depth of n with constant-size frames uses O(n) stack space. Do not ignore this memory simply because the program does not allocate an explicit array.

The common trap

Do not infer complexity from the number of loops visible in the code. A loop that repeatedly halves a search interval runs O(log n) times. A nested inner loop with a fixed bound of 10 adds a constant factor, producing O(n) total work.

State what n represents and which case you are analyzing. Worst-case, average-case, and amortized costs answer different questions.

Keep these ideas

Complexity describes growth, not exact seconds. Count dominant operations. Add consecutive work and multiply genuinely nested work. Include recursive stack memory. Compare both time and space when selecting a solution.

Module 02

Arrays & linked lists

A collection's layout determines what it can do efficiently. Arrays place elements in contiguous positions; linked lists connect separately stored nodes through references. Understanding this difference lets you choose deliberately between fast indexed access and flexible structural changes.

Two ways to organize a sequence

An array stores a sequence in contiguous memory. Given a base address, element size, and index, the location of an element can be calculated directly. This is why accessing a known index takes O(1) time.

A linked list stores values in nodes. Each node contains a value and a reference to another node. The nodes need not be adjacent in memory. To find the element at index k, a singly linked list follows k links from its head.

Both represent ordered sequences. Their different layouts produce different operation costs.

The cost of an array operation

Access by index is O(1), but searching an unsorted array is O(n) in the worst case. Inserting near the beginning of a packed array requires moving subsequent elements to make space. Deleting an element similarly shifts later values to close the gap.

A dynamic array maintains a capacity that can exceed its current length. Appending while capacity remains is O(1). An append that triggers reallocation can cost O(n). With geometric capacity growth, a sequence of appends has O(1) amortized cost per append.

Amortized cost describes a sequence of operations; it does not mean every individual append is constant time.

Worked example: insert into an array

Generated practice example: insert 25 at index 2 of [10, 20, 30, 40].

1. Ensure the array has space for one more element.
2. Shift 40 from index 3 to index 4.
3. Shift 30 from index 2 to index 3.
4. Write 25 at index 2.
5. The result is [10, 20, 25, 30, 40].

The shift proceeds from right to left so that values are not overwritten before being moved. Inserting near the start of a large array can require O(n) shifts.

A linked list changes the trade-off

In a singly linked list, a node stores a value and a next reference. The final node's next reference is null. The head identifies the first node.

If the predecessor node is already known, insertion can update a constant number of links. First point the new node to the successor, then point the predecessor to the new node. This link update is O(1). Finding that predecessor may still take O(n).

A doubly linked list stores both previous and next references. This supports movement in both directions at the cost of additional memory and more link updates.

Worked example: link a new node

Generated practice example: insert node B between nodes A and C in A -> C.

1. Create B with the desired value.
2. Set B.next = A.next, so B points to C.
3. Set A.next = B.
4. The chain is now A -> B -> C.

Reversing the two assignments carelessly can lose the original successor. Handle empty lists and insertion at the head explicitly.

Choose based on your operations

Prefer an array when frequent index access, compact storage, and sequential traversal dominate. Consider linked structures when structural updates at already-known nodes dominate.

The statement 'linked-list insertion is always O(1)' hides the cost of locating the insertion point. Similarly, 'array append is O(1)' should specify whether it means amortized cost. State the assumptions before comparing structures.

Keep these ideas

Arrays calculate locations; linked lists follow references. Array indexing is O(1). Unsorted search is O(n) in both structures. Array insertion may shift elements. Linked insertion changes links in O(1) after the predecessor is known. Choose for the operations your application performs most often.

Module 03

Stacks & queues

Stacks and queues are abstract data types defined by the order in which elements leave. A stack returns the most recently added element. A queue returns the earliest waiting element. These simple rules support function calls, undo history, buffering, and graph traversal.

A stack remembers the most recent item

A stack follows last in, first out (LIFO). Push adds an item to the top, pop removes the top item, and peek observes it without removal. Popping an empty stack is an underflow condition and must be handled.

An implementation can use a dynamic array or linked nodes. The abstract rule stays the same. Array-backed push can be amortized O(1), while linked push updates the head in O(1).

Worked example: a stack trace

Generated practice example: start with an empty stack. Push A, push B, pop, then push C.

1. After push A: [A].
2. After push B: [A, B], with B at the top.
3. Pop returns B and leaves [A].
4. Push C leaves [A, C].

The next pop returns C. The most recently added surviving item leaves first.

A queue respects arrival order

A queue follows first in, first out (FIFO). Enqueue adds at the rear; dequeue removes from the front. Peek observes the front.

An array implementation that shifts every element on dequeue costs O(n) for that operation. A circular buffer uses front and rear indices to reuse positions and avoid these shifts. A linked implementation with both head and tail references supports O(1) enqueue and dequeue.

Worked example: queue order

Generated practice example: enqueue A, B, and C, then dequeue twice. The first dequeue returns A, and the second returns B. C remains at the front.

If D is enqueued next, the queue becomes [C, D]. D cannot overtake C under the FIFO rule. This makes a queue appropriate for processing tasks in arrival order.

Match the ordering to the task

Undo operations naturally use a stack because the latest action is undone first. Breadth-first graph traversal uses a queue because earlier-discovered vertices are processed before later layers.

Do not confuse an abstract data type with its storage implementation. A queue is not necessarily a linked list, and a stack is not necessarily an array. The ordering contract comes first.

Module 04

Trees & hierarchies

Trees describe hierarchical relationships. A root leads to child nodes, and every non-root node has one parent. Binary trees limit each node to two children. Binary search trees add an ordering rule that can make search efficient when the tree stays balanced.

Read the shape of a tree

A rooted tree has a distinguished root. A node with no children is a leaf. A path follows connected nodes; depth counts edges from the root. The height is the largest number of edges on a path from the node to a leaf.

A tree with n nodes has n - 1 edges. This relationship reflects its connected, acyclic structure. Do not confuse a general tree with a binary tree, which allows at most two children per node.

The search-tree invariant

In a binary search tree with distinct keys, every key in a node's left subtree is smaller than that node's key, and every key in its right subtree is larger. The rule applies to whole subtrees, not just immediate children.

Compare the desired key with the current node. Move left if smaller and right if larger. Stop at a match or an empty child. The cost is O(h), where h is tree height. A balanced tree has O(log n) height; a skewed tree can have O(n) height.

Worked example: search a tree

Generated practice example: a search tree has root 8, left child 3, and right child 12. Node 3 has right child 6. Search for 6.

1. Compare 6 with 8; move left.
2. Compare 6 with 3; move right.
3. Reach 6 and return the matching node.

The comparisons follow the ordering rule and avoid visiting the entire right subtree.

Traversal changes the visit order

Preorder visits the node before its left and right subtrees. Inorder visits the left subtree, then the node, then the right subtree. Postorder visits both subtrees before the node.

For a binary search tree with distinct keys, inorder traversal yields sorted keys. This sorted result depends on the search-tree invariant; inorder traversal of an arbitrary binary tree need not be sorted.

Balance is an assumption

A plain binary search tree does not automatically stay balanced. Inserting already sorted keys can produce a chain. Search then degrades to O(n).

When stating O(log n) search, specify the balance assumption or use a data structure that maintains it. Tree shape and key ordering are different properties.

Module 05

Graphs & networks

Graphs represent entities and the relationships connecting them. A vertex might be a city, a person, or a course topic. Edges encode roads, friendships, or prerequisites. The graph in this application is itself a practical example: modules are vertices and prerequisite links are directed edges.

Relationships become edges

A graph contains a set of vertices and a set of edges. An undirected edge connects two vertices symmetrically. A directed edge has a start and an end. A weighted edge stores a value such as distance or cost.

A path follows a sequence of adjacent vertices. A cycle returns to its starting vertex. Directed prerequisite graphs should be acyclic if every prerequisite must be completed before a dependent module.

Choose a representation

An adjacency list stores each vertex's neighbors. It uses O(V + E) space and works well for sparse graphs. An adjacency matrix stores a V by V table. It uses O(V^2) space and supports direct edge lookup in O(1).

Choose based on graph density and required operations. A matrix may waste memory when most pairs of vertices have no edge.

Breadth-first search explores layers

Breadth-first search uses a queue. Mark the start vertex visited and enqueue it. Repeatedly dequeue a vertex and enqueue each unvisited neighbor, marking each neighbor when enqueued.

With adjacency lists, the traversal runs in O(V + E). In an unweighted graph, BFS finds shortest paths measured by the number of edges. This guarantee does not extend to arbitrary weighted edges.

Worked example: BFS layers

Generated practice example: A connects to B and C; B connects to D; C also connects to D. Start at A.

1. Visit A and enqueue B and C.
2. Dequeue B; discover and enqueue D.
3. Dequeue C; D is already marked, so do not enqueue it again.
4. Dequeue D.

The layers are A, then B and C, then D. Neighbor ordering can change the order within one layer.

Depth-first search follows a path

Depth-first search uses recursion or an explicit stack. It explores a path as far as possible before backtracking. A visited set prevents endless revisiting in graphs with cycles.

BFS and DFS both explore connectivity in O(V + E) with adjacency lists. Their exploration orders serve different purposes; DFS does not generally produce shortest paths.

Module 06

Searching & sorting

Searching finds a desired item; sorting establishes an order. The right approach depends on whether data is already sorted, which operations dominate, and how much extra memory is available. These algorithms bring complexity analysis and data representation together.

Begin with linear search

Linear search compares the target with each item until a match is found or the collection ends. It works on unsorted data and takes O(n) time in the worst case.

For one search in a small unsorted collection, linear search may be a sensible choice. Sorting first has a cost and is not automatically worthwhile for a single query.

Binary search needs order

Binary search maintains a sorted search interval. Compare the target with the middle value and discard the half that cannot contain it. Repeat until a match is found or the interval is empty.

The interval halves each step, giving O(log n) comparisons. Efficient implementation assumes fast access to the midpoint, as in an array. Binary search does not work correctly on an arbitrary unsorted array.

Worked example: halve the interval

Generated practice example: search for 18 in [2, 5, 9, 12, 18, 24, 30].

1. The middle value is 12. Since 18 is larger, keep the right interval.
2. The middle of [18, 24, 30] is 24. Keep its left interval.
3. Compare with 18 and return the matching index.

Use consistent inclusive or half-open bounds. Mixing conventions is a common source of off-by-one errors.

Merge sort divides and combines

Merge sort divides a sequence into halves, recursively sorts each half, then merges the sorted halves. Each recursion level performs O(n) merge work and there are O(log n) levels, giving O(n log n) time.

A conventional array implementation uses O(n) auxiliary storage for merging. Stability can be preserved by taking an equal element from the left half first.

Choose with the whole workload in mind

Repeated searches can justify the upfront cost of sorting. Compare that cost with the number of expected queries. Memory limits may also change which sorting method is appropriate.

State preconditions explicitly: binary search requires sorted data, and complexity claims depend on the representation and implementation. A faster asymptotic bound is one part of the decision.